



COMP9321:

Data services engineering

Week 5: GraphQL

(A quick Overview)

Term 3, 2025

By Morty Al-Banna, CSE UNSW

What is GraphQL

“GraphQL is a **query language for APIs** and a runtime for fulfilling those queries with your existing data. GraphQL provides a complete and understandable description of the data in your API, gives clients **the power to ask for exactly what they need and nothing more**, makes it easier to evolve APIs over time, and enables powerful developer tools.”

Where Did GraphQL Come From

- Created by Meta (Facebook) team in 2012
- Made Opensource in 2015
- Currently managed by GraphQL Foundation, hosted by the non-profit Linux Foundation.

REST API vs GraphQL

	REST	GraphQL
What is it?	REST is a set of rules that defines structured data exchange between a client and a server.	GraphQL is a query language, architecture style, and set of tools to create and manipulate APIs.
Best suited for	REST is good for simple data sources where resources are well defined.	GraphQL is good for large, complex, and interrelated data sources.
Data access	REST has multiple endpoints in the form of URLs to define resources.	GraphQL has a single URL endpoint.
Data returned	REST returns data in a fixed structure defined by the server.	GraphQL returns data in a flexible structure defined by the client.
How data is structured and defined	REST data is weakly typed. So, the client must decide how to interpret the formatted data when it is returned.	GraphQL data is strongly typed. So, the client receives data in predetermined and mutually understood formats.
Error checking	With REST, the client must check if the returned data is valid.	With GraphQL, invalid requests are typically rejected by schema structure. This results in an autogenerated error message.

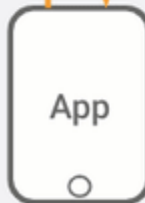
HTTP request

POST /graphql

```
query {  
  user (id: 1) {  
    id  
  }  
}
```

HTTP response

```
{  
  "user": {  
    "id": 1,  
  }  
}
```



GraphQL API call

GraphQL Fields and Arguments

- The Response mimics the fields requested in the query.
- Fields might come from single or multiple resources

Query	Response
<pre>hero (id: "1000") { name # Queries can have comments! friends { name } }</pre>	<pre>{ "hero": { "name": "R2-D2", "friends": [{ "name": "Luke Skywalker" }, { "name": "Han Solo" }, { "name": "Leia Organa" }] } }</pre>

GraphQL Queries, Mutations, and Subscriptions

- The operation type is either query, mutation, or subscription and describes what type of operation you're intending to do.
- Think about Query being Read, Mutation being create or update, and Subscriptions being a notification of updates.

```
mutation CreateReviewForEpisode($ep: Episode!  
  createReview(episode: $ep, review: $review  
    stars  
    commentary  
  )  
}
```

VARIABLES

```
{  
  "ep": "JEDI",  
  "review": {  
    "stars": 5,  
    "commentary": "This is a great movie!"  
  }  
}
```

```
{  
  "data": {  
    "createReview": {  
      "stars": 5,  
      "commentary": "This is a great movie!"  
    }  
  }  
}
```

Advantages/Disadvantages of GraphQL

Advantages	Disadvantages
Efficient data Fetching	Giving the Power to the requester can be dangerous
Strongly Typed Scheme (which can be an advantage in some cases)	Schema Design Overhead
Reduced number of endpoints	Complexity overhead (especially if you are used to REST)
Easier Evolution	Caching Challenges
Better Performance	Error Handling Complexity

Useful Resources

- <https://www.howtographql.com/basics/0-introduction/>
- <https://graphql.org/learn/>
- https://www.youtube.com/watch?v=783ccP__No8

Q&A